

Data manipulation with pandas

Lesson 1: Inspecting data

- **Introduction to DataFrames:** You discovered that DataFrames are used to store and manipulate tabular data, where data is organized into rows and observations and columns represent variables.
- **Inspecting DataFrames:** You learned how to use methods like `.head()`, `.info()`, `.shape`, and `.describe()` to explore and understand the structure and contents of a DataFrame. These methods help in getting a quick overview of the DataFrame, including the number of rows and columns, data types of each column, and some basic statistics for numerical columns.
- **DataFrame Components:** You found out that a DataFrame consists of three components: `.values`, `.columns`, and `.index`. You learned that `.values` is a two-dimensional NumPy array of the DataFrame's values, `.columns` contains the column names, and `.index` holds the row labels.

Here's a snippet of code you worked with to explore these components:

```
import pandas as pd

# Print the values of homelessness
print(homelessness.values)

# Print the column index of homelessness
print(homelessness.columns)

# Print the row index of homelessness
print(homelessness.index)
```

This lesson laid the foundation for further exploration and manipulation of DataFrames in pandas, preparing you for more complex data analysis tasks.

The goal of the next lesson is to introduce techniques for manipulating and exploring data using pandas, including sorting, subsetting, and filtering DataFrames.

Lesson 2: Aggregating data with pd

- **Mean and Median:** You discovered how to calculate the mean and median of a DataFrame column, which represent the average and the midpoint value of the dataset,

respectively. For example, `sales["weekly_sales"].mean()` calculates the average weekly sales.

- **Minimum and Maximum:** You explored finding the minimum and maximum values in a DataFrame, useful for understanding the data's range. This was applied to both numeric and date columns.
- **Custom Aggregations:** Through the `.agg()` method, you learned to apply custom functions for more tailored summary statistics, such as calculating specific percentiles.
- **Cumulative Statistics:** You practiced calculating cumulative sums and maximums with `.cumsum()` and `.cummax()`, which help in analyzing data trends over time.

Example of calculating cumulative statistics

```
sales_1_1["cum_weekly_sales"] = sales_1_1["weekly_sales"].cumsum()
```

```
sales_1_1["cum_max_sales"] = sales_1_1["weekly_sales"].cummax()
```

Sorting data

- **Sorting DataFrames:** You discovered how to reorder rows using the `.sort_values()` method. By specifying a column name, you can sort the DataFrame based on that column. For descending order, set `ascending=False`. To sort by multiple columns, pass a list of column names to the method.
- **Subsetting Columns:** You learned that to select specific columns from a DataFrame, you use square brackets `[]` with the column name or names inside. For multiple columns, you need two sets of square brackets: the inner set for the list of column names, and the outer set for subsetting the DataFrame.
- **Filtering Rows:** You explored how to subset rows using conditions. For numerical data, relational operators like `>` can be used to filter rows. For categorical data, you can use `==` to filter by specific values or `.isin()` for multiple values.

Here's an example of subsetting rows based on a condition:

```
dogs[dogs["height_cm"] > 60]
```

Counting Data

- **Counting Categorical Data:** You discovered how to summarize categorical data using the `value_counts` method. This method counts the occurrences of each unique value in a column, which is particularly useful for understanding the distribution of categorical variables.

Removing Duplicates: You learned to use the `drop_duplicates` method to remove duplicate rows based on specific criteria. By specifying the `subset` argument, you can focus on unique combinations of columns, ensuring accurate counts. For example, to drop duplicate

store and type combinations, you used:

```
store_types = sales.drop_duplicates(subset=["store", "type"])
```

- **Counting with Proportions:** Besides counting occurrences, you learned to calculate proportions to understand the relative frequencies of categories using the `normalize=True` argument with `value_counts`. This helps in identifying the share of each category within the total.
- **Practical Application:** Through exercises, you applied these concepts to real-world scenarios, such as counting the number of visits by dog breeds to a vet office and analyzing sales data to understand store and department distributions.

You learned about aggregating data in pandas DataFrames, focusing on calculating summary statistics for grouped data. This is crucial for analyzing subsets within your data to uncover patterns or differences across categories.

- **Grouped Summary Statistics:** You discovered how to use the `.groupby()` method to group data before applying summary functions like `.mean()`. For instance, grouping dogs by color and calculating the average weight per group simplifies the process and reduces the risk of errors compared to doing this manually for each group.

Calculating Without `.groupby()`: You also explored calculating grouped summary statistics without `.groupby()`, using a real-world example of calculating the proportion of sales by store type at Walmart. This involved subsetting the DataFrame for each store type and calculating total sales for each subset.

```
# Calc total weekly sales
```

```
sales_all = sales["weekly_sales"].sum()
```

```
# Subset for type A stores, calc total weekly sales
```

```
sales_A = sales[sales["type"] == "A"]["weekly_sales"].sum()
```

```
# Subset for type B stores, calc total weekly sales
```

```
sales_B = sales[sales["type"] == "B"]["weekly_sales"].sum()
```

```
# Subset for type C stores, calc total weekly sales
```

```
sales_C = sales[sales["type"] == "C"]["weekly_sales"].sum()
```

```
# Get proportion for each type
```

```
sales_propn_by_type = [sales_A, sales_B, sales_C] / sales_all
```

```
print(sales_propn_by_type)
```

- **Multiple Grouped Summaries:** You learned to perform more complex aggregations by grouping data by multiple columns and using the `.agg()` method with NumPy functions to calculate multiple statistics at once, such as min, max, mean, and median for different variables across grouped data.

Lesson 3: Indexes and Pivot tables

- **Creating Pivot Tables:** You discovered how to use the `.pivot_table()` method in pandas to group data and calculate summary statistics. For instance, to calculate the mean weight of dogs by color, you would specify the `values` as the column to summarize and the `index` as the column to group by.
- **Custom Summary Functions:** You learned that the default aggregation function is the mean, but you can use the `aggfunc` argument to apply other functions, like NumPy's `median` function, for different summary statistics.
- **Multiple Summary Statistics:** By passing a list of functions to `aggfunc`, you can obtain multiple statistics at once, such as mean and median weights.
- **Grouping by Multiple Variables:** You explored how to group data by more than one variable by passing a second variable name to the `columns` argument, allowing for more detailed analysis.
- **Handling Missing Values:** The `fill_value` argument was introduced as a way to replace missing values (NaNs) with a specified value, such as zero, to avoid gaps in your data analysis.
- **Adding Margins:** You saw how setting `margins=True` adds additional rows and columns to the pivot table, showing the mean of all values in a row or column, providing a high-level summary of the data.

Here's an example code snippet from the lesson:

```
import pandas as pd

import numpy as np

# Example of creating a pivot table

pivot_table = df.pivot_table(values='Weight', index='Color',
aggfunc=np.median)
```

- Understanding that DataFrames consist of data, row indexes, and column indexes. You learned how to set a column as an index using the `set_index` method, which helps in aligning and quickly accessing the data.

- You discovered how to reset an index back to its default integer values using `reset_index`, with an option to drop the index column if it's no longer needed.
- The lesson highlighted the use of `.loc[]` for subsetting rows based on index values, making code cleaner and more readable. For example, `temperatures_ind.loc["Moscow"]` retrieves all rows for Moscow from a DataFrame indexed by city.
- You delved into multi-level or hierarchical indexing, which allows for indexing on multiple columns. This is particularly useful for nested categorical variables and can be set up with `set_index` by passing a list of column names.
- Finally, you learned how to sort DataFrames by index values using `sort_index`, enhancing data organization and retrieval. This can be done on multiple levels of the index, either in ascending or descending order.

Example of setting and resetting an index

```
temperatures_ind = temperatures.set_index("city")
print(temperatures_ind.reset_index())
```

Slicing and Subsetting With `.loc` `.iloc`

- **Slicing Lists and DataFrames:** You discovered that slicing allows you to select consecutive elements using `start:end` syntax, where `start` is included but `end` is excluded. For DataFrames, this requires sorted indexes.
- **Using `.loc` and `.iloc`:** You learned that `.loc` is used for label-based indexing, where you specify the name of the rows and columns you want to select. For example, `df.loc["row1":"row3", "column1":"column3"]` selects rows 'row1' through 'row3' and columns 'column1' through 'column3'. Conversely, `.iloc` is used for position-based indexing, similar to slicing lists, but applicable to both rows and columns.
- **Multi-level Indexing:** You explored how to slice DataFrames with multi-level indexes by specifying tuples for the `.loc` method, allowing you to target specific sub-sections of your DataFrame.
- **Slicing Columns:** You found out that slicing can also be applied to columns by passing column names to the `.loc` method, and how to select specific rows and columns simultaneously.
- **Time Series Slicing:** You saw the power of pandas in handling time series data, learning to slice by date ranges using the `.loc` method. This included slicing by partial dates, which is particularly useful for time series analysis.

Here's an example code snippet from the lesson:

Subset rows from Pakistan, Lahore to Russia, Moscow

```
print(temperatures_srt.loc[("Pakistan", "Lahore"):(("Russia", "Moscow"))])
```

Lesson 4: Visualizing Data

- Creating pivot tables using the `pivot_table` method, where you specify the values to aggregate, and the indices and columns to group by. The default aggregation function is mean.
- Adding a new column to a DataFrame based on date components, such as extracting the year from a date to use in a pivot table.
- Subsetting pivot tables using the `.loc[]` method combined with slicing to select specific ranges of data. This is useful for drilling down into specific segments of your data.
- Calculating summary statistics on pivot tables, such as the mean temperature for each year or city. You can calculate these statistics across rows or columns by specifying the `axis` argument.

For example, to create a pivot table of average temperatures by country and city versus year and then see the result, you would use:

```
# Add a year column to temperatures
temperatures["year"] = temperatures["date"].dt.year

# Pivot avg_temp_c by country and city vs year
temp_by_country_city_vs_year = temperatures.pivot_table("avg_temp_c",
index=["country", "city"], columns="year")

# See the result
print(temp_by_country_city_vs_year)
```

Histograms

- **Histograms** help visualize the distribution of a numeric variable. You created a histogram for dog heights using `dog_pack['height'].hist()` and displayed it with `plt.show()`, learning how the number of bins can affect the granularity of the distribution visualized.

- **Bar plots** are useful for showing the relationship between a categorical and a numeric variable. You calculated the average weight of dogs by breed and then created a bar plot to visualize these averages.
- **Line plots** are ideal for visualizing changes over time. You looked at how a Labrador named Sully's weight changed over the year by plotting weight against date.
- **Scatter plots** allow you to examine the relationship between two numeric variables. You plotted dog height versus weight to see that taller dogs tend to weigh more.
- You also explored how to manipulate data for plotting, such as grouping data and calculating sums or means, and how to make plots more readable by adjusting aspects like label rotation and plot transparency.

Here's an example of creating a bar plot to visualize avocado sales by size:

```
# Import matplotlib.pyplot with alias plt
import matplotlib.pyplot as plt

# Get the total number of avocados sold of each size
nb_sold_by_size = avocados.groupby("size")["nb_sold"].sum()

# Create a bar plot of the number of avocados sold by size
nb_sold_by_size.plot(kind="bar")

# Show the plot
plt.show()
```

Missing Values and Replacing Missing values

- **Identifying Missing Values:** You discovered how to use the `.isna()` method to check for missing values in a DataFrame. This method returns a DataFrame of Booleans indicating the presence of missing values.
- **Summarizing Missing Values:** You learned to apply `.any()` and `.sum()` methods after `.isna()` to identify columns with missing values and count them, respectively. This is crucial for understanding the extent of missing data.
- **Visualizing Missing Data:** Through a practical example, you visualized missing values using a bar plot. This visual representation helps in quickly identifying the columns with the most significant number of missing values.
- **Removing Missing Values:** You explored how to remove rows with missing values using the `.dropna()` method. This approach is straightforward but might not always be the best option as it can lead to the loss of valuable data.
- **Replacing Missing Values:** Finally, you learned about replacing missing values with a specific value using the `.fillna()` method. This technique is particularly useful when you want to retain as much data as possible by making assumptions about the missing values.

Here's a snippet of code you worked with, demonstrating how to visualize missing values in a DataFrame:

```
import matplotlib.pyplot as plt

# Check individual values for missing values
print(avocados_2016.isna())

# Bar plot of missing values by variable
avocados_2016.isna().sum().plot(kind="bar")

plt.show()
```

Creating DataFrames

- **Dictionaries in Python:** You discovered that dictionaries store data as key-value pairs, which can be used to create DataFrames. For example, `{"Title": "Charlotte's Web"}` shows a simple dictionary with a key "Title" and its value.
- **Creating DataFrames from Lists of Dictionaries:** This method builds DataFrames row by row. Each dictionary in the list represents a row in the DataFrame, with keys as column names and values as row data.
- **Creating DataFrames from Dictionaries of Lists:** This approach constructs DataFrames column by column, where each key-value pair in the dictionary represents a column, keys being the column names and values being lists of column data.
- **Practical Application:** You applied what you learned by creating a DataFrame for avocado sales data using both methods. For instance, using the list of dictionaries method:

```
avocados_list = [
    {"date": "2019-11-03", "small_sold": 10376832, "large_sold": 7835071},
    {"date": "2019-11-10", "small_sold": 10717154, "large_sold": 8561348},
]

avocados_2019 = pd.DataFrame(avocados_list)
```